

Missing Colon in For-Loop

- The for loop within the `from_roman` function does not have a colon following its parentheses. The colon is required by Python to run the loop, so without it, there is a syntax error. I fixed this error by re-adding the colon to the function line, allowing this line in the program to run.
- Original code: `for i in range(1, len(roman_numeral))`
- Updated code: `for i in range(1, len(roman_numeral)):`
- Type of Error: Syntax
- Test case:
 - Input: I, V
 - Expected output: "I is smaller than V"
 - Actual Output: `SyntaxError: expected ':'`

Missing Parentheses in Print-Function

- The print statement under the main function is missing a parenthesis. This is another syntax error preventing the print statement from executing, and therefore the entire code. Print statements must have parentheses around the statement itself in order to run, so with one missing, it was unable to run. I was able to fix this syntax error by adding a closing parenthesis to the print statement.
- Original code: `print(f'{num1} is {compare} {num2}')`
- Updated code: `print(f'{num1} is {compare} {num2}')`
- Type of Error: Syntax
- Test case:
 - Input: X, X
 - Expected output: "X is equal to X"
 - Actual Output: `SyntaxError: '(' was never closed`

Incorrect Comparison Between a and b

- There is an incorrect comparison relation between a and b in the `compare_roman_numerals` function. The comparison shows that if a is not equal to b, 0 is returned. Therefore, any comparison between a and b where a is not less than b will return a and b as equal. This is a semantic error, as the program technically runs fine, but it's not the output we want. I fixed this by changing the unequal comparison to equals.

- Original code: if a < b:
 return -1
if a != b:
 return 0
return 1
- Updated code: if a < b:
 return -1
if a == b:
 return 0
return 1
- Type of Error: Semantic
- Test case:
 - Input: V, I
 - Expected output: V is larger than I
 - Actual Output: V is equal to I

Error Subtracting From Decimal Value

- When trying to subtract from the decimal value, the code uses 'symbol' instead of 'symbols.' 'Symbol' is a variable that represents a Roman numeral character, not a number value. Trying to index a string with a string leads to a type error, and cannot be subtracted from the decimal value. I fixed it by changing 'symbol' to 'symbols.'
- Original code: decimal_value -= symbol[roman_numeral[-1 - i]]
- Updated code: decimal_value -= symbols[roman_numeral[-1 - i]]
- Type of Error: Runtime error
- Test case:
 - Input: IV, V
 - Expected output: "IV is smaller than V"
 - Actual Output: TypeError: string indices must be integer

Incorrect Values Assigned In the Dictionary

- The values for the keys 'D' and 'C' are incorrectly assigned in the symbols dictionary. 'C' and 'D' should denote 100 and 500, respectively, but instead, it is the other way around. To fix this, I will switch the keys for each assigned value in the dictionary to obtain the correct values.
- Original code: symbols = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'D': 100, 'C': 500, 'M': 1000}

- Updated code: symbols = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
- Type of Error: Semantic Error
- Test case:
 - Input: C, D
 - Expected output: C is smaller than D
 - Actual Output: C is larger than D

Incorrect Placement of the return decimal value

- The values for the decimal value is returned too early because it is placed inside the for loop rather than outside of it. Despite it being a loop, the value is going to be returned after one iteration because once something is returned, the function code will stop. I fixed this issue by placing the return statement for the decimal value inside the function, but outside the loop.

```
def from_roman(roman_numeral):
    '''In the Roman numeral system, the symbols I, V, X, L, C, D, and M stand resp
    symbols = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
    if is_empty(roman_numeral):
        return 0
    if len(roman_numeral) == 1:
        return symbols[roman_numeral[0]]
    symbol = roman_numeral[-1]
    decimal_value = symbols[symbol]
    previous_symbol = symbol
    for i in range(1, len(roman_numeral)):
        if symbols[roman_numeral[-1 - i]] >= symbols[pre (variable) i: int
            decimal_value += symbols[roman_numeral[-1 - i]]
        else:
            decimal_value -= symbols[roman_numeral[-1 - i]]
            previous_symbol = roman_numeral[-1 - i]
    return decimal_value
```

- Original code:

```
def from_roman(roman_numeral):
    '''In the Roman numeral system, the symbols I, V, X, L, C, D, and M stand resp
    symbols = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
    if is_empty(roman_numeral):
        return 0
    if len(roman_numeral) == 1:
        return symbols[roman_numeral[0]]
    symbol = roman_numeral[-1]
    decimal_value = symbols[symbol]
    previous_symbol = symbol
    for i in range(1, len(roman_numeral)):
        if symbols[roman_numeral[-1 - i]] >= symbols[previous_symbol]:
            decimal_value += symbols[roman_numeral[-1 - i]]
        else:
            decimal_value -= symbols[roman_numeral[-1 - i]]
            previous_symbol = roman_numeral[-1 - i]
    return decimal_value
```

- Updated code:
- Type of Error: Semantic Error
- Test case:
 - Input: XIV, X

- Expected output: XIV is larger than X
- Actual Output: XIV is smaller than X

Incorrect Placement of the previous symbol variable

- In order for the for loop in the from roman function to work properly, the loop must iterate through the roman numeral correctly. Originally, the previous symbol was only updated if there was a subtraction from the decimal value. However, this update of the previous symbol should be happening after every iteration, so to fix this, I moved the previous symbol variable outside the else statement, but still kept it in the loop.
- Original code:

```
symbols = for i in range(1, len(roman_numeral)):
    if symbols[roman_numeral[-1 - i]] >= symbols[previous_symbol]:
        decimal_value += symbols[roman_numeral[-1 - i]]
    else:
        decimal_value -= symbols[roman_numeral[-1 - i]]
        previous_symbol = roman_numeral[-1 - i]
```
- Updated code:

```
symbols = for i in range(1, len(roman_numeral)):
    if symbols[roman_numeral[-1 - i]] >= symbols[previous_symbol]:
        decimal_value += symbols[roman_numeral[-1 - i]]
    else:
        decimal_value -= symbols[roman_numeral[-1 - i]]
    previous_symbol = roman_numeral[-1 - i]
```
- Type of Error: Semantic Error
- Test case:
 - Input: MCMXCIX, MDCCCCLXXXVIII
 - Expected output: MCMXCIX is equal to MDCCCCLXXXVIII
 - Actual Output: MCMXCIX is larger than MDCCCCLXXXVIII